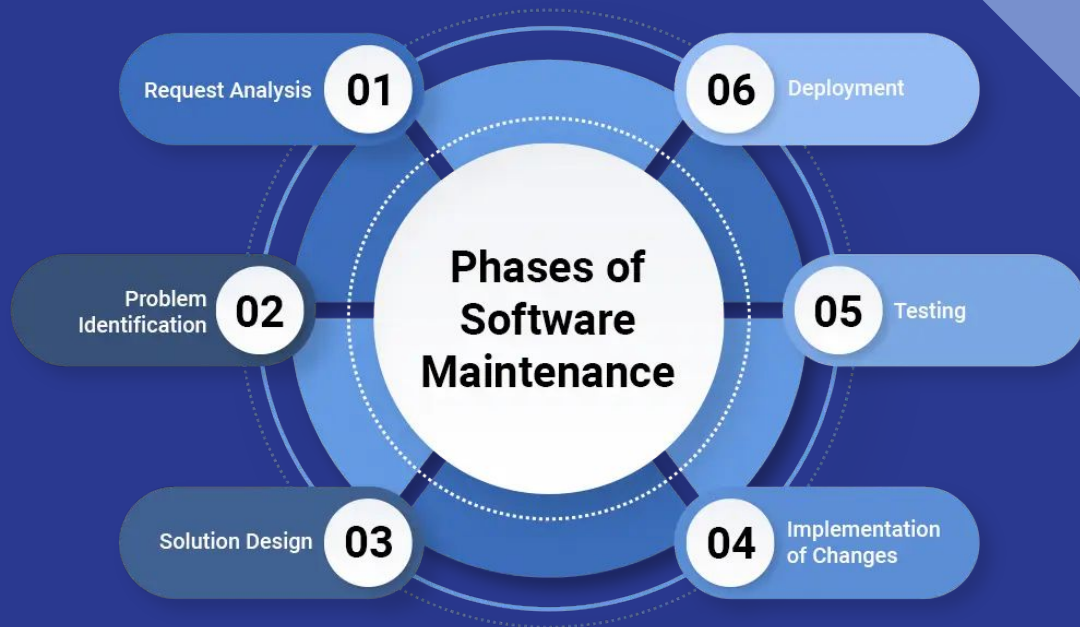


Software Maintenance

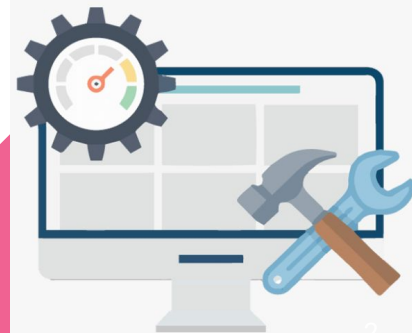
การบำรุงรักษาซอฟต์แวร์



What is Software Maintenance ?

การบำรุงรักษาซอฟต์แวร์ (software Maintenance) คือการเปลี่ยนแปลงซอฟต์แวร์หลังจากการส่งมอบ เพื่อแก้ไขปัญหาหรือข้อผิดพลาดเพื่อนำมาปรับปรุงให้มีประสิทธิภาพ หรือ การดัดแปลงซอฟต์แวร์ให้มีความเหมาะสมกับสภาพแวดล้อมที่เปลี่ยนแปลงไป

It is a critical part of the [software development life cycle \(SDLC\)](#) and is necessary to ensure that the software continues to meet the needs of the users over time.





การบำรุงรักษาผลิตภัณฑ์ไม่ได้เป็นเพียงแค่การแก้ไขบั๊ก (Bug) หรือการเขียนโค้ดเพิ่มเติมเท่านั้น แต่เป็นกระบวนการที่ครอบคลุมถึงการปรับปรุงและแก้ไขซอฟต์แวร์อย่างรอบด้าน

การสร้างระบบซอฟต์แวร์อาจใช้เวลาเพียง 2 ปี ในขณะที่การบำรุงรักษานั้นอาจกินเวลายาวนานถึง 15 ถึง 20 ปี

Software Maintenance Phases

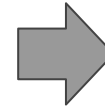


Ref: <https://www.codzgarage.com/blog/software-maintenance-life-cycle/>

ทำไมการบำรุงรักษาซอฟต์แวร์จึงมีความจำเป็น?

มีเหตุผลหลายประการที่ซอฟต์แวร์ของคุณจำเป็นต้องได้รับการดูแล และบำรุงรักษา ซึ่งได้แก่:

- **การแก้ไขบั๊กและข้อผิดพลาดต่างๆ** (Correcting bugs and faults)
- **การปรับปรุงซอฟต์แวร์** ให้รองรับกับความท้าทายและสภาพแวดล้อมใหม่ๆ
- **การปรับเปลี่ยนการออกแบบ** (Changing the design)
- **การเชื่อมต่อการทำงานร่วมกับระบบอื่นๆ** (Integrating with other systems)
- **การป้องกันปัญหาทางเทคนิค** ที่อาจเกิดขึ้นไว้ล่วงหน้า
- **การปลดระวางซอฟต์แวร์** (Retiring software) หรือการยกเลิกการใช้งานเมื่อหมดอายุขัย



ประเภทของการบำรุงรักษาซอฟต์แวร์



ประเภทของการบำรุงรักษาซอฟต์แวร์

1. Corrective Maintenance (การบำรุงรักษาเพื่อแก้ไขข้อผิดพลาด)

คือการแก้ไขปัญหาที่เกิดขึ้นหลังจากที่ซอฟต์แวร์ถูกส่งมอบให้ผู้ใช้งานแล้ว

- **เป้าหมาย:** แก้ไข "บั๊ก" (Bugs) หรือข้อผิดพลาดในการทำงานที่ไม่เป็นไปตามความต้องการ
- ข้อบกพร่องหรือบั๊กเหล่านี้สามารถถูกตรวจพบได้ผ่านวิธีการต่างๆ เช่น ข้อเสนอแนะจากผู้ใช้งาน (User Feedback), การเฝ้าติดตามระบบ (Monitoring), หรือการทดสอบ"



Corrective Maintenance คือการแก้สิ่งที่ **"ทำงานผิดพลาด"** หรือ **"ไม่ปลอดภัย"** ให้กลับมาถูกต้องตามทีออกแบบไว้

ตัวอย่าง Corrective Maintenance

1. ข้อผิดพลาดร้ายแรง (Critical Bugs / Emergency Fixes)

เป็นปัญหาที่ทำให้ระบบทำงานไม่ได้ ต้องรีบแก้ไขทันที

- **ตัวอย่าง:** ผู้ใช้งานกดปุ่ม "ชำระเงิน" ในแอปฯ แล้วแอปฯ **ปิดตัวเอง (Crash)** ทันที
- **ตัวอย่าง:** เว็บไซต์ลงทะเบียนเรียนล่ม ไม่สามารถเข้าใช้งานได้เนื่องจากมีคนเข้าพร้อมกันมากเกินไป (ต้องแก้ Code เรื่อง Memory Leak)
- **ตัวอย่าง:** พบช่องโหว่ความปลอดภัยที่ทำให้แฮกเกอร์เข้าถึงข้อมูลส่วนตัวลูกค้าได้ (ต้องรีบทำการ update security version ใหม่ทันที) >> Hotfix (การแก้ปัญหาแบบเร่งด่วน ทันทีทันใด)

2. ข้อผิดพลาดทางตรรกะและการคำนวณ (Logic & Calculation Errors)

ระบบทำงานได้ ไม่พัง แต่ผลลัพธ์ที่ได้ออกมา ผิด

- **ตัวอย่าง:** ระบบคำนวณภาษีมูลค่าเพิ่ม (VAT) เป็น 0% แทนที่จะเป็น 7% ในบางบิล
- **ตัวอย่าง:** โปรแกรมตัดเกรดคำนวณคะแนนรวมผิดพลาด ทำให้นักเรียนที่ได้ 80 คะแนนได้เกรด B แทนที่จะเป็น A
- **ตัวอย่าง:** รายงานยอดขายประจำเดือนดึงข้อมูลมาผิดช่วงเวลา (เช่น ดึงของเดือนที่แล้วมาแสดงแทนเดือนปัจจุบัน)

ตัวอย่าง Corrective Maintenance (ต่อ)

3. ข้อผิดพลาดของข้อมูลและการแสดงผล (Data & Display Errors)

ข้อมูลถูกต้อง แต่การนำเสนอผิดพลาด

- **ตัวอย่าง:** วันที่ในระบบแสดงผลผิดรูปแบบ (เช่น แสดงเป็น ปี/วัน/เดือน ทำให้ผู้ใช้อ่านผิด)
- **ตัวอย่าง:** ชื่อ-นามสกุล ของผู้ใช้งานที่มีสระวรรณยุกต์ยาวๆ แสดงผลซ้อนทับกันจนอ่านไม่ออก
- **ตัวอย่าง:** รูปสินค้าในหน้าเว็บไม่ตรงกับชื่อสินค้าที่แสดง

4. ข้อผิดพลาดด้านความเข้ากันได้ (Compatibility Bugs)

- **ตัวอย่าง:** เว็บไซต์แสดงผลได้ปกติบน Chrome แต่เมื่อเปิดบน Safari แล้วเมนูหายไป
- **ตัวอย่าง:** ปุ่ม "ตกลง" ตกขอบหน้าจอเมื่อเปิดในมือถือรุ่นที่มีหน้าจอขนาดเล็ก

ประเภทของการบำรุงรักษาซอฟต์แวร์ (ต่อ)

2. Adaptive Maintenance (การบำรุงรักษาเพื่อให้ปรับตัวได้)

คือการปรับปรุงซอฟต์แวร์เพื่อให้สามารถทำงานได้ในสภาพแวดล้อมที่เปลี่ยนแปลงไป

- **เป้าหมาย:** ให้ซอฟต์แวร์ยังคงใช้งานได้เมื่อมีการเปลี่ยน Hardware, ระบบปฏิบัติการ (OS), หรือกฎระเบียบทางธุรกิจใหม่ๆ (ปรับตัวตาม Environment/Hardware/Policy)
 - **ตัวอย่าง:** การอัปเดตแอปฯ ให้รองรับ iOS หรือ Android เวอร์ชันใหม่, การย้ายฐานข้อมูลจาก Server เดิมขึ้นสู่ Cloud, การอัปเดตปลั๊กอิน (Plugins) และไลบรารี (Libraries) ให้เป็นเวอร์ชันล่าสุด

Adaptive Maintenance: หลักการคือ "ซอฟต์แวร์เราดีอยู่แล้ว แต่โลกภายนอกเปลี่ยนไป เราเลยต้องเปลี่ยนตามเพื่อให้ยังใช้งานได้"

ตัวอย่าง Adaptive Maintenance

1. การเปลี่ยนแปลงด้านกฎหมายและระเบียบข้อบังคับ (Regulatory Changes)

- **ตัวอย่าง:** รัฐบาลประกาศเปลี่ยนอัตราภาษีมูลค่าเพิ่ม (VAT) หรือเปลี่ยนกฎการคำนวณภาษีหัก ณ ที่จ่าย
 - **สิ่งที่ต้องทำ:** แก้ไขสูตรคำนวณในโปรแกรมบัญชีให้ตรงกับกฎหมายใหม่

2. การเปลี่ยนแปลงด้านฮาร์ดแวร์และระบบปฏิบัติการ (Hardware & OS Updates)

- **ตัวอย่าง:** Apple ปล่อย iOS เวอร์ชันใหม่ (เช่น iOS 18) แล้วพอนต์ในแอปพลิเคชัน เพี้ยน หรือพีเจอร์บางอย่างใช้ไม่ได้
 - **สิ่งที่ต้องทำ:** อัปเดตโค้ดของแอปฯ ให้รองรับโครงสร้างใหม่ของ iOS 18

3. การเปลี่ยนแปลงของซอฟต์แวร์ภายนอกที่เชื่อมต่อด้วย (Third-party Integration)

- **ตัวอย่าง:** ระบบ Login ผ่าน Facebook (Facebook Login) มีการเปลี่ยนเวอร์ชัน API ใหม่ และยกเลิกเวอร์ชันเก่า
 - **สิ่งที่ต้องทำ:** แก้ไขโค้ดส่วนยืนยันตัวตนให้เรียกใช้ API เวอร์ชันใหม่ ไม่งั้นผู้ใช้จะ Login ไม่ได้

4. การย้ายแพลตฟอร์ม (Platform Migration)

- **ตัวอย่าง:** ย้ายฐานข้อมูลจาก MySQL ไปใช้ PostgreSQL หรือย้ายจาก Server ที่ตัวเอง (On-premise) ขึ้นไปบน Cloud (AWS/Azure/Google Cloud)
 - **สิ่งที่ต้องทำ:** แก้ไขโค้ดส่วนการเชื่อมต่อฐานข้อมูล (Connection String) และคำสั่ง SQL บางคำสั่งที่แตกต่างกันระหว่างสองระบบ

ประเภทของการบำรุงรักษาซอฟต์แวร์ (ต่อ)

3. Preventive Maintenance (การบำรุงรักษาเพื่อป้องกัน)

คือการปรับปรุงโครงสร้างภายในของซอฟต์แวร์เพื่อลดความเสี่ยงที่จะเกิดปัญหาในอนาคต (มักเรียกว่า Software Re-engineering)

- **เป้าหมาย:** ทำให้ซอฟต์แวร์ดูแลรักษาง่ายขึ้นในระยะยาว ลดความซับซ้อนของโค้ด
- รวมถึงการเฝ้าดูระบบอย่างใกล้ชิด การอัปเดตส่วนประกอบต่างๆ เช่น ปลั๊กอิน (Plugins) และไลบรารี (Libraries) รวมถึงการนำมาตรการป้องกันมาใช้ เช่น การสำรองข้อมูล (Backup)
 - **ตัวอย่าง:** การทำ Code Refactoring (จัดระเบียบโค้ดใหม่ให้อ่านง่ายขึ้น), การปรับปรุงเอกสารประกอบ (Documentation), หรือการลบตัวแปรที่ไม่ได้ใช้ออกไป

หลักการสำคัญของข้อนี้คือ **"กันไว้ดีกว่าแก้"** (Prevention is better than cure) เป็นการกระทำเพื่อลดความเสี่ยงที่ระบบจะพังในอนาคต หรือเพื่อให้ดูแลรักษาง่ายขึ้น

ตัวอย่าง Preventive Maintenance

1. Code Refactoring (การจัดระเบียบโค้ดใหม่)

- **สถานการณ์:** โปรแกรมเมอร์คนเก่าเขียนโค้ดไว้ซับซ้อนมาก (Spaghetti Code) มีการ Copy-Paste โค้ดชุดเดิมซ้ำๆ หลายที่ โปรแกรมยังทำงานได้ปกติ แต่ถ้าต้องแก้ตรงนี้ ต้องตามไปแก้อีกหลายจุด
- **การทำ Preventive:**
 - ยุบรวมโค้ดที่ซ้ำซ้อนให้เป็นฟังก์ชันเดียว (Extract Function)
 - เปลี่ยนชื่อตัวแปรจาก `a`, `b` ให้สื่อความหมาย เช่น `userId`, `totalPrice`
 - **ผลลัพธ์:** ไม่ได้ฟีเจอร์ใหม่ แต่ถ้าในอนาคตต้องมาแก้ bug จะทำได้เร็วขึ้นและลดโอกาสผิดพลาด



2. Database Optimization & Archiving (การดูแลฐานข้อมูล)

- **สถานการณ์:** ระบบเก็บ Log การใช้งานทุกวัน วันละ 1 แสนรายการ เมื่อเวลาผ่านไป 3 ปี ข้อมูลในตารางมีเป็นร้อยล้านแถว ทำให้การค้นหาเริ่มช้าลงเรื่อยๆ (ระบบยังไม่ crash แต่ใกล้จะ Time out)
- **การทำ Preventive:**
 - **Data Archiving:** เขียนสคริปต์ให้ย้ายข้อมูลเก่ากว่า 1 ปี ไปเก็บในถังเก็บข้อมูลแยก (Cold Storage) หรือแยก Table
 - **Re-indexing:** สั่งจัดเรียงสารบัญข้อมูลใหม่ (Rebuild Index) เพื่อให้ค้นหาข้อมูลเร็วขึ้น
 - **ผลลัพธ์:** ป้องกันไม่ให้ระบบ Database ล่มเพราะข้อมูลเต็ม หรือ Query ช้าจนค้าง

ประเภทของการบำรุงรักษาซอฟต์แวร์ (ต่อ)

4. Perfective Maintenance (การบำรุงรักษาเพื่อความสมบูรณ์)

คือการปรับปรุงซอฟต์แวร์ให้ดียิ่งขึ้น เพื่อตอบสนองความต้องการของผู้ใช้ที่เพิ่มขึ้น หรือเพื่อให้ทำงานได้มีประสิทธิภาพกว่าเดิม

- **เป้าหมาย:** เพิ่มฟีเจอร์ใหม่ (new features), ปรับปรุง UI ให้ใช้งานง่ายขึ้น หรือปรับจูน (tune) ให้โปรแกรมทำงานเร็วขึ้น
- เป็นการปรับปรุงประสิทธิภาพการทำงานบางอย่าง แม้ว่าไม่มีข้อผิดพลาดเกิดขึ้น
 - **ตัวอย่าง:** การเพิ่มปุ่ม "Dark Mode", การเพิ่มระบบชำระเงินผ่าน QR Code ตามที่ลูกค้าเรียกร้องเรียน
 - จัดหน้ารายงานให้อ่านง่ายขึ้น ปรับปรุงหน้าโอนเงินให้ใช้งานง่ายขึ้น เพิ่มความสามารถในรับค้นหารายการอุปกรณ์ เช่น เดิมค้นหาได้จากชื่อผู้ยืม เพิ่มเป็นให้ค้นหาด้วยวันที่ยืมอุปกรณ์ เป็นต้น

คอนเซปต์หลักของข้อนี้คือ "ซอฟต์แวร์เดิมก็ทำงานได้ดีอยู่แล้ว ไม่ได้พัง ไม่ได้มีบั๊ก แต่เราต้องการทำให้ 'ดีขึ้น' หรือ 'เก่งขึ้น' เพื่อเอาใจผู้ใช้งาน"

ตัวอย่าง Perfective Maintenance

1. การเพิ่มฟีเจอร์ใหม่ (Adding New Features) เป็นรูปแบบที่พบบ่อยที่สุด คือเมื่อผู้ใช้มีความต้องการใหม่ๆ เพิ่มขึ้นจากตอนแรก

- **สถานการณ์:** แอปพลิเคชันธนาคารเดิมโอนเงินได้ปกติ แต่ลูกค้าบ่นว่าไม่ยากจำเลขบัญชีเพื่อน
 - สิ่งที่ทำ (Perfective): เพิ่มฟีเจอร์ "**โอนเงินผ่านเบอร์โทรศัพท์ (PromptPay)**" หรือเพิ่มฟีเจอร์ "**สแกน QR Code**" เข้าไป
- **สถานการณ์:** ระบบส่งงานนักศึกษา (LMS) เดิมทำได้แค่ส่งไฟล์
 - สิ่งที่ทำ (Perfective): เพิ่มฟีเจอร์ "**ตรวจ Plagiarism (การคัดลอกผลงาน) อัตโนมัติ**" หรือเพิ่มปุ่ม "**Export คะแนนเป็น Excel**" ให้อาจารย์สะดวกขึ้น

2. การปรับปรุงหน้าตาและการใช้งาน (UI/UX Improvements) เพื่อให้ผู้ใช้รู้สึกว่าย่อยง่ายขึ้น สวยขึ้น ทันสมัยขึ้น

- **สถานการณ์:** โปรแกรมบัญชีหน้าตาดูโบราณ ปุ่มกดยาก ต้อง click หลายครั้งกว่าจะบันทึกข้อมูลสำเร็จ
 - สิ่งที่ทำ (Perfective): ออกแบบหน้าจอใหม่ (Redesign) ให้สะอาดตา, เพิ่ม **Shortcuts (คีย์ลัด)** ให้กดบันทึกได้ไวขึ้น, หรือเพิ่ม **Dark Mode** เพื่อถนอมสายตา
- **สถานการณ์:** เว็บไซต์ขายของ ต้องกรอกที่อยู่ใหม่ทุกครั้งที่ซื้อ
 - สิ่งที่ทำ (Perfective): เพิ่มระบบ "**จดจำที่อยู่ล่าสุด**" หรือ "**Login with Facebook/Google**" เพื่อลดขั้นตอนการกรอกข้อมูล



สรุปความแตกต่างให้เห็นภาพ

เปรียบเทียบ "การบำรุงรักษาท่อน้ำ"

- **Corrective:** "ท่อน้ำแตก น้ำนองพื้นแล้ว" -> ซ่อมท่อ
- **Adaptive:** "การประปาเปลี่ยนแรงดันน้ำใหม่" -> ปรับวาล์วให้รองรับ
- **Perfective:** "อยากได้น้ำไหลแรงขึ้นไว้อาบเสร็จเร็ว" -> เปลี่ยนปั้มน้ำใหม่
- **Preventive:** "ท่อยังดีอยู่ แต่เห็นสนิมเริ่มเกาะ" -> ขัดสนิมและทาสีกันสนิม (ทำก่อนที่มันจะรั่ว)

เปรียบเทียบกับ "การดูแลบ้าน"

1. **Corrective:** หลังคารั่ว ฝนตกน้ำหยด -> อุดรอยรั่วบนหลังคา
2. **Adaptive:** ลูกโตขึ้นต้องการห้องส่วนตัว หรือน้ำท่วมบ่อยขึ้น -> กั้นห้องเพิ่ม หรือ ดัดบ้านให้สูงขึ้นหนีน้ำ
3. **Perfective:** บ้านก็อยู่สบายดี แต่อยากได้ครัวใหม่สวยๆ -> รีโนเวทห้องครัว ปูพื้นกระเบื้องลายใหม่
4. **Preventive:** ปลูกยังไม่ขึ้นบ้าน -> จ้างบริษัทกำจัดปลวกมาฉีดน้ำยา ป้องกันไว้ก่อน



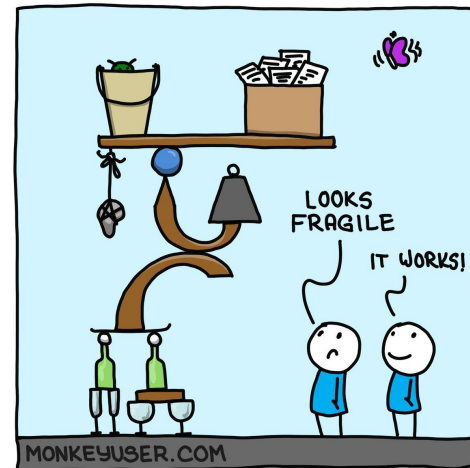
เลือกอะไรดี การบำรุงรักษา (Maintenance) หรือ การพัฒนาใหม่ (Replacement/Redevelopment)

1. คุณภาพของซอฟต์แวร์เดิม (Technical Quality)

- **ความเปราะบางของโค้ด (Code Fragility):** ถ้าแก้จุดหนึ่งแล้วไปพังอีกหลายจุด (Spaghetti Code) การบำรุงรักษาจะเริ่มไม่คุ้ม
- **เทคโนโลยีที่ใช้ (Obsolescence):** ภาษาโปรแกรมหรือ Framework ที่ใช้ยังได้รับการสนับสนุนอยู่หรือไม่? หากไม่สามารถหาโปรแกรมเมอร์มาดูแลต่อไม่ได้แล้ว (เช่น ภาษา COBOL หรือระบบเก่ามากๆ) การพัฒนาใหม่คือทางเลือกที่ปลอดภัยกว่า
- **ความซับซ้อน (Cyclomatic Complexity):** หากค่า $V(G)$ สูงมากเกินไปจนจะควบคุมได้ การรีเขียนใหม่อาจประหยัดเวลาดีกว่าการพยายามทำความเข้าใจโค้ดเดิม

2. ค่าใช้จ่าย (Economics)

- **Maintenance Cost vs. Development Cost:** ให้ลองคำนวณค่าแรงที่เสียไปกับการแก้บั๊กรายวันเทียบกับงบลงทุนสร้างระบบใหม่
- **กฎ 80/20:** ถ้าเราต้องใช้เวลา 80% ของงบประมาณเพียงเพื่อรักษาให้ระบบเดิม "ยังทำงานได้" (Keep the lights on) โดยไม่มีฟีเจอร์ใหม่เพิ่มขึ้นเลย เป็นสัญญาณว่าควรพัฒนาใหม่ได้แล้ว



3. ความต้องการทางธุรกิจ (Business Requirements)

- **ความสามารถในการขยายตัว (Scalability):** ระบบเดิมรองรับผู้ใช้งานที่เพิ่มขึ้นในอนาคตได้ไหม
- **การเชื่อมต่อ (Integration):** ระบบเก่าสามารถเชื่อมต่อกับเทคโนโลยีสมัยใหม่ (เช่น Cloud, AI, หรือ API ต่างๆ) ได้สะดวกหรือไม่
- **Time-to-Market:** หากการเพิ่มฟีเจอร์ใหม่ในระบบเดิมใช้เวลา 6 เดือน แต่ระบบใหม่อาจใช้เวลาสร้าง 8 เดือนแล้วเพิ่มฟีเจอร์ได้ทุกสัปดาห์ การลงทุนใหม่จะคุ้มค่าง่ายกว่าในระยะยาว

4. ข้อมูลและทรัพย์สินทางปัญญา (Data & Knowledge)

- **ความเข้าใจในระบบเดิม:** มีเอกสารประกอบ (Documentation) หรือทีมงานที่รู้จักเรื่อง Business Logic เดิมหรือไม่ ถ้า "คนทำไม่อยู่ ความรู้หาย" การบำรุงรักษาจะกลายเป็นการเดาสุ่ม ซึ่งมีความเสี่ยงเป็นอย่างมาก



เลือกอะไรดี การบำรุงรักษา (Maintenance) หรือ การพัฒนาใหม่ (Replacement/Redevelopment)

ตารางเปรียบเทียบการตัดสินใจ

ปัจจัย	เลือก "บำรุงรักษา" (Maintain)	เลือก "พัฒนาใหม่" (Replace)
สภาพโค้ด	โครงสร้างยังดี แก้ไขง่าย	ซับซ้อนเกินเยียวยา แก้จุดหนึ่งพังจุดหนึ่ง
เทคโนโลยี	ยังทันสมัย มีคนดูแลได้	ตกรุ่น หาคนทำยาก ไม่มี Security Patch
งบประมาณ	มีงบจำกัด เน้นประคองระบบ	มีงบลงทุน และต้องการผลตอบแทนระยะยาว
ความยืดหยุ่น	ธุรกิจไม่เปลี่ยนแปลงมาก	ต้องการฟีเจอร์ที่ระบบเดิมทำไม่ได้แน่นอน

References :

1. **Software Maintenance - Software Engineering**
<https://www.geeksforgeeks.org/software-engineering-software-maintenance/>
2. **Importance of Software Maintenance in Software Engineering**
<https://cleancommit.io/blog/importance-of-software-maintenance-in-software-engineering/>
3. **What is Reverse-Engineering in Software Engineering?**
<https://www.scaler.com/topics/reverse-engineering-in-software-engineering/>
4. **Technique for Software Maintenance** <https://www.csd.uwo.ca/~kkontogi/publications/books/b1-2011.pdf>
5. **Software Maintainace cost** <https://maddevs.io/customer-university/software-maintenance-costs/>